Язык программирования C++

Обработка ошибок Исключительные ситуации

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Обработка ошибок

В процессе работы программы возможно возникновение **непредвиденных ситуаций**:

- нехватка памяти и других системных ресурсов;
- ошибки ввода-вывода;
- некорректные данные, поступившие от пользователя;
- нарушение целостности данных (поврежден файл с данными);
- некорректные параметры функций.

Хорошо спроектированная программа должна уметь **обнаруживать, сигнализировать и обрабатывать** данные ситуации:

- сообщить об ошибке пользователю;
- попытаться исправить ошибки;
- при невозможности дальнейшей работы – сохранить данные и завершить работу.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Чтение данных из потока

Обнаружить ошибку и попытаться её исправить можно **локально** (в месте её появления), если для этого имеется вся необходимая информация. Рассмотрим такой случай на примере **потока std::cin**.

Для получения данных от пользователя и вставки их в переменную используется операция >> (**операция извлечения**).

Когда пользователь вводит символы в ответ на запрос операции извлечения, то они попадают в **буфер** внутри потока cin. Буфер представляет собой часть оперативной памяти, в которой введённые символы хранятся до момента их извлечения в переменные.

При использовании операции извлечения выполняются следующие шаги:

- Если в буфере уже есть данные, то они используются для извлечения.
- Если буфер пустой, то ожидается ввод данных от пользователя. Когда пользователь нажимает “Enter”, в буфер помещаются все введённые символы включая завершающий ‘\n’.
- Операция >> извлекает из буфера максимально возможное количество данных, которые можно поместить в переменную. При этом игнорируются начальные пробельные символы.
- Все данные, которые не могут быть извлечены, остаются в буфере до следующего извлечения.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример чтения данных из потока

Извлечение считается **успешным**, если хотя бы один символ был считан из буфера.

Например, имеется следующий код:

```
int x;
cin >> x;
```

Если пользователь введёт с клавиатуры “5a” и нажмёт “Enter”, то будет считан символ ‘5’, преобразован в тип int и помещён в переменную **x**. В буфере останется “a\n” до следующего извлечения.

Извлечение будет считаться **проваленным** (fail), если входные данные не соответствуют типу переменной. Например, если пользователь введёт с клавиатуры ‘b’ и нажмёт “Enter”, то извлечение провалится, поскольку ‘b’ не может быть приведено к целому числу.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример программы (1)

Рассмотрим пример программы, моделирующей простейший калькулятор, без проверки вводимых значений и обработки ошибок.

```
#include <iostream>
using namespace std;

double getDouble()
{
  cout << "Введите число: ";
  double x;
  cin >> x;
  return x;
}

char getOperator()
{
  cout << "Введите операцию (+,-,*,/): ";
  char op;
  cin >> op;
  return op;
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример программы (2)

```
void printResult(double x, char operation, double y) {
  switch (operation) {
  case '+':
    cout << x << " + " << y << " = " << x + y << '\n';
    break;
  case '-':
    cout << x << " - " << y << " = " << x - y << '\n';
    break;
  case '*':
    cout << x << " * " << y << " = " << x * y << '\n';
    break;
  case '/':
    cout << x << " / " << y << " = " << x / y << '\n';
    break;
  }
}

int main() {
  double x = getDouble();
  char operation = getOperator();
  double y = getDouble();
  printResult(x, operation, y);
  return 0;
}
```

```
Введите число: 5
Введите операцию (+,-,*,/): *
Введите число: 7
5 * 7 = 35
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Типы ошибок при текстовом вводе данных

При использовании программы с прошлых слайдов, возможны три типа ошибок, которые могут возникнуть из-за неправильного ввода данных:

1. Извлечение данных **успешно, но входные данные бессмысленны** (например, ввод символа ‘k’ в качестве математической операции) .
2. Извлечение данных **успешно, но пользователь ввёл дополнительные символы** (например, введено “* hello” в качестве математической операции).
3. Извлечение данных **провалено** (например, введено ‘q’ в качестве числа).

Рассмотрим каждый из них.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

1. Успешно, но бессмысленно

Простейший случай. Рассмотрим такое выполнение программы:

```
Введите число: 5
Введите операцию (+,-,*,/): k
Введите число: 7
```

Мы ожидаем, что пользователь введёт один из 4 символов математических операций, но он вводит ‘k’. Поскольку ‘k’ является обычным символом, то cin без вопросов записывает его в переменную **op**. Однако программа не ожидает наличие такого символа, поэтому никогда не выдаст никакого ответа.

Решением проблемы является **проверка вводимой информации**:

1. Проверить, введена ли ожидаемая информация.
2. Если да, то вернуть её из функции.
3. Если нет, то вывести сообщение об ошибке и запросить повторный ввод. Для этого часто используется бесконечный цикл.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

1. Успешно, но бессмысленно. Решение

Перепишем функцию getOperator(), добавив проверку:

```
char getOperator() {
  while (true) { // Пока не введено корректно
    cout << "Введите операцию (+,-,*,/): ";
    char operation;
    cin >> operation;
    switch (operation) {
    case '+':
    case '-':
    case '*':
    case '/':
      return operation; // возвращаем из функции
    default: // если ввели неправильную информацию
      cerr << "Ввод некорректный.\n";
    }
  } // Пробуем ввести снова
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

2. Успешно, но с лишним

Рассмотрим ситуацию, когда пользователь при первом же запросе введёт все данные:

```
Введите число: 5*7
Введите операцию (+,-,*,/): Введите число: 5 * 7 = 35
```

Результат вычислений корректный, но вывод выглядит плохо.

Когда пользователь ввел символы “5*7”, то все они были помещены в буфер. При первом вызове операции >> из буфера будет считан символ ‘5’ и помещён в переменную **x**. При этом в буфере останутся символы “*7\n”. При втором вызове из буфера будет считан символ ‘*’. Запроса на ввод данных пользователем не произойдёт. При третьем вызове операции >> из буфера считается символ ‘7’.

Лучшим решением будет **игнорирование всех лишних символов** с помощью метда cin.ignore() до символа перевода на новую строку.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

2. Успешно, но с лишним. Решение

Оформим игнорирование лишних символов в виде отдельной функции:

Переменная **streamsize** хранит максимальный размер буфера.

```
void ignoreLine()
{
  cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

double getDouble()
{
  cout << "Введите число: ";
  double x;
  cin >> x;
  ignoreLine(); // Очищаем буфер
  return x;
}
```

numeric_limits<streamsize>::max() возвращает максимальное значение, которое можно сохранить в переменной **streamsize**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

3. Извлечение данных провалено

Если при первом же запросе ввести некорректное значение:

```
Введите число: a
Введите операцию (+,-,*,/): Ввод некорректный.
Введите операцию (+,-,*,/): Ввод некорректный.
Введите операцию (+,-,*,/): Ввод некорректный.
```

Когда пользователь вводит символ ‘a’, то он помещается в буфер. После этого происходит попытка приведения символа к типу double и записи в переменную **x**. Этого не происходит, из-за чего внутри cin устанавливается флаг fail в значение true. После этого все последующие запросы на ввод данных считаются провальными, что приводит к бесконечному циклу.

Для решения проблемы необходимо **проверить значение флага fail** с помощью метода cin.fail() или использования приведения потока к булевому значению: !cin.

После этого нужно вернуть поток в нормальное состояние методом cin.clear() и очистить буфер.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

3. Извлечение данных провалено. Решение

Перепишем функцию getDouble(), добавив проверку :

```
double getDouble() {
  while (true) { // Пока не введено корректно
    cout << "Введите число: ";
    double x;
    cin >> x;
    if (!cin) { // Проверяем провал чтения
      cin.clear(); // Переводим в нормальное состояние
      ignoreLine(); // Очищаем буфер
    }
    else {
      ignoreLine();
      return x; // Выходим из функции
    }
  }
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Невозможность локального исправления

Во многих ситуациях **локальное исправление возникшей ошибки невозможно** в связи с недостатком информации.

Например, если у нас есть функция, вычисляющая значение по формуле, включающей квадратный корень:

```
double mySqrt(double x)
{
  if (x < 0.0)
    // Нужно как-то сообщить, что произошла ошибка
  return sqrt(x);
}
```

Если функции было передано отрицательное число, то получить корректный результат не получится. «Молча» вернуть 0.0 в качестве результата в большинстве случаев также не является подходящим вариантом.

В связи с этим встаёт вопрос об информировании вызывающей функции о произошедшей внештатной ситуации. Для этого в C++ введено понятие **исключений**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Понятие исключения

Исключение (исключительная ситуация) (exception) – особая, часто аномальная, ситуация, возникающая в программе на этапе исполнения и нарушающая обычный ход вычислительного процесса.

Например, деление на ноль или обращение по несуществующему адресу памяти.

Механизм исключений предназначен только для событий, которые происходят в результате работы самой программы и указываются явным образом.

Исключения возникают тогда, когда некоторая часть программы не смогла сделать то, что от нее требовалось. При этом другая часть программы может попытаться сделать что-нибудь иное.

Таким образом, исключения позволяют логически разделить вычислительный процесс на две части:

1) генерация информации о возникновении аварийной ситуации, которая не может быть разрешена локально;

2) обработка ошибок, обнаруженных в других местах.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Общий механизм обработки исключений

Место, в котором может произойти ошибка, должно входить в **контролируемый блок** – составной оператор, перед которым записано ключевое слово **try**.

Реализация обработки исключительных ситуаций:

- Обработка исключения начинается с появления ошибки. Функция, в которой она возникла, генерирует исключение. Для этого используется ключевое слово **throw** с параметром, определяющим **тип исключения**. *Параметр может быть константой, переменной или объектом и используется для передачи информации об исключении его обработчику.*

- Отыскивается соответствующий обработчик исключения – составной оператор, перед которым записано ключевое слово **catch**, а тип параметра совпадает с типом исключения. Ему передается управление.

- Если обработчик исключения не найден, вызывается стандартная функция **terminate()**, которая вызывает функцию **abort()**, аварийно завершающую текущий процесс. Можно установить собственную функцию завершения процесса.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обобщенная схема обработки исключений

```
void funct_a()
{
  try
  {
    // любой код, который может сгенерировать исключение
    if(условие_генерации_исключения)
        throw T(); // Генерация исключения типа T (может быть любым)
  }
  catch (T X)
  {
    // обработчик исключения X типа T, которое могло быть
    // ранее сгенерировано внутри предыдущего блока try
  }
  // Обработчики других исключений предыдущего блока try
  catch (...)
  {
    // обработчик любого исключения предыдущего блока try
  }
  funct_x(); // функция, которая будет вызвана после выполнения
             // try-блока или обработчиков исключения
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Последовательность событий при обработке исключений

1. Код выполняется стандартно вне контролируемого блока **try**.
2. Управление переходит в контролируемый блок **try**.
3. Какое-то выражение в этом блоке приводит к возникновению ошибки.
4. Производится генерация исключения **throw**.
5. Управление передается обработчику событий **catch**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис исключений

Служебное слово **try** служит для обозначения блока кода, в котором может генерироваться исключение (**try-блока**):

```
try
{
  // операторы
}
```

Все функции, *прямо или косвенно* вызываемые из try-блока, также считаются ему принадлежащими.

Генерация (порождение) исключения происходит по ключевому слову throw, которое употребляется либо с параметром, либо без него:

```
throw [ выражение ];
```

Тип выражения, стоящего после **throw**, определяет тип порождаемого исключения.

При генерации исключения выполнение текущего блока прекращается, и происходит поиск соответствующего обработчика и передача ему управления.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Синтаксис исключений (продолжение)

Непосредственно за try-блоком располагаются обработчики исключений. Они начинаются с ключевого слова **catch**, за которым в скобках следует тип обрабатываемого исключения. Можно записать один и более обработчиков в соответствии с типами обрабатываемых исключений.

```
catch (тип имя)
{
  // тело обработчика
}
```

Применяется, когда имя параметра используется в теле обработчика для выполнения каких-либо действий (например, вывода информации об исключении).

```
catch (тип)
{
  // тело обработчика
}
```

Не предполагает использования информации об исключении, играет роль только его тип.

```
catch (…)
{
  // тело обработчика
}
```

Многоточие вместо параметра обозначает, что обработчик перехватывает все исключения.
Этот обработчик следует помещать после всех остальных.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Действия, выполняемые с момента генерации исключения до завершения его обработки

- При генерации исключения (**throw X**) создается временный объект – копия **X**. С этой копией будет работать выбранный далее обработчик; она существует до тех пор, пока обработка исключения не будет завершена.
- Для всех других объектов try-блока, созданных к этому моменту, освобождается память; при этом для объектов классов вызывается деструктор. То же делается и для уже созданных подобъектов – полей классов. Этот процесс называют **раскручиванием стека**.
- Если в списке обработчиков этого try-блока найден подходящий, то выполняются его операторы; затем выполнение программы продолжается с оператора, расположенного за последним обработчиком этого try-блока.
- Если в списке данного try-блока не нашлось подходящего обработчика, то поиск продолжается в динамически объемлющих try-блоках (т.е. внешних try-блоках по отношению к текущему (вложенному) try-блоку); при этом процесс раскручивания стека продолжается.
- Если подходящего обработчика так и не нашлось, то вызывается функция terminate() и выполнение программы прекращается.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример

```
#include <iostream>
#include <fstream>
class Hello { // информирует о своем создании и уничтожении

public:
  Hello() { cout << "Конструктор. Hello!" << endl; }
  ~Hello() { cout << "Деструктор. Bye!" << endl; }
};

void f1() {
  ifstream ifs("\\INVALID\\FILE\\NAME"); // Открываем файл
  if (!ifs) {
    cout << "Генерируем исключение" << endl;
    throw "Ошибка при открытии файла";
  }
}

void f2() {
  Hello H; // Создаем локальный объект
  f1(); // Вызываем функцию, генерирующую исключение
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример (продолжение)

```
int main() {
  system("chcp 1251");
  try {
    cout << "Входим в try-блок" << endl;
    f2();
    cout << "Выходим из try-блока" << endl;
  }
  catch(int i) {
    cout << "Вызван обработчик int, исключение - " << i << endl;
    return -1;
  }
  catch(const char * p) {
    cout << "Вызван обработчик const char*, " << endl
         << "  исключение - " << p << endl;
    return -1;
  }
  catch(...) {
    cout << "Вызван обработчик всех исключений" << endl;
    return -1;
  }
  return 0; // Все обошлось благополучно
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример (продолжение)

Результаты выполнения программы:

```
Входим в try-блок
Конструктор. Hello!
Генерируем исключение
Деструктор. Bye!
Вызван обработчик const char *,
  исключение - Ошибка при открытии файла
```

Обратите внимание, что **сообщение «Выходим из try-блока» не было выведено.**

После порождения исключения **был вызван деструктор локального объекта**, хотя управление из функции f1() было передано обработчику, находящемуся в функции main().

Таким образом, механизм исключений позволяет корректно уничтожать объекты при возникновении ошибочных ситуаций.

Поэтому **выделение и освобождение ресурсов полезно оформлять в виде классов**, конструктор которых выделяет ресурс, а деструктор освобождает.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Классы исключений

Использование классов в качестве типа объектов исключений имеет ряд преимуществ:

- Возможность хранения подробной информации об исключении.
- Возможность использования полиморфизма:
  - Оператор catch ловит исключения не только указанного типа, но и всех производных от него типов:

```
try
{
        throw DerivedException;
}
catch (const BaseException & e)
{
        /* */
}
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Группировка исключений

Организация исключений в виде иерархий может иметь большое значение для надежности кода.

Для примера рассмотрим возможную обработку исключений в библиотеке математических функций без механизма группировки:

```
void g ()
{
  try { /*....*/ }
  // обработка переполнения значения переменной
  catch (Overflow) { /*...*/ }
  // обработка деления на ноль
  catch (Zerodivide) { /*...*/ }
}
```

Недостатки этого подхода:

- Необходимо перечислить все исключения, однако программист может легко забыть указать исключение в этом списке.
- При введении нового исключения в этой библиотеке, необходимо было бы «дописывать» каждый фрагмент кода, обрабатывающий все математические исключения.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример (продолжение)

Результаты выполнения программы:

```
Входим в try-блок
Конструктор. Hello!
Генерируем исключение
Деструктор. Bye!
Вызван обработчик const char *,
  исключение - Ошибка при открытии файла
```

Обратите внимание, что **сообщение «Выходим из try-блока» не было выведено.**

После порождения исключения **был вызван деструктор локального объекта**, хотя управление из функции f1() было передано обработчику, находящемуся в функции main().

Таким образом, механизм исключений позволяет корректно уничтожать объекты при возникновении ошибочных ситуаций.

Поэтому **выделение и освобождение ресурсов полезно оформлять в виде классов**, конструктор которых выделяет ресурс, а деструктор освобождает.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Производные исключения

Зачастую исключения перехватывается обработчиком его базового класса:

```
class Matherr { //...
  virtual void print() const
  { cerr << "Математическая ошибка"; }
};

// Исключение для случая переполнения переменной типа int
class Int_overflow: public Matherr {
  string op;
  int a1, a2;
public:
  Int_overflow (const string& p, int a, int b)
  { op = p;   a1 = a;   a2 = b; }
  virtual void print() const
  { cerr << op << '(' << a1 << ', ' << a2 << ')'; }
};

void f() {
  try { g(); }
  catch (Matherr & m) { /*...*/ }
}
```

!!! Правильно будет именно по ссылке

Необходимо перехватывать исключения по ссылке, чтобы вызывался обработчик не только базового, но и производных классов.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Производные исключения (продолжение)

Во избежание потери информации, можно использовать указатели или ссылки:

```
int add (int x, int y) {
  if ((x>0 && y>0 && x>INT_MAX-y)
    || (x<0 && y<0 && x<INT_MIN-y))
    throw Int_overflow ("+", x, y);
  return x+y;
}

void f() {
  try {
    int i1=add(1, 2);
    int i2=add(INT_MAX, -2);
    int i3=add(INT_MIN, -2);
  }
  catch (Matherr & m) {
    //...
    m.print();
  }
}
```

INT_MAX / INT_MIN – максимальное / минимальное значение, которое может храниться в переменной типа int.

Также можно использовать такое выражение:
`numeric_limits<int>::max()`
`numeric_limits<int>::min()`

Последний вызов add() приведет к исключению, которое вызовет Int_overflow::print().

Если исключение перехватывалось бы по значению, то был бы вызван метод Matherr::print().

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Повторная генерация исключений

Если обработчик не может полностью обработать исключение, то он делает то, что может, после чего вновь генерирует исключение. Таким образом, ошибка может быть обработана в наиболее подходящем месте. Это происходит и в том случае, когда информация, требуемая для корректной обработки ошибки, распределена по нескольким обработчикам:

```
void h() {
  try {
    /*код, который может привести к математическим ошибкам*/
  }
  catch (Matherr) {
    // если может обработать полностью
    if (can_handle_it_completely) {
      // обработка Matherr
      return;
    } else {
      // делает то, что можно сделать здесь
      throw; // повторная генерация исключения
    }
  }
}
```

Факт повторной генерации отмечается отсутствием операнда у throw.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Порядок записи обработчиков

Из-за того, что производные исключения могут быть перехвачены обработчиками, предназначенными для более чем одного вида исключений, чрезвычайно важен порядок записи обработчиков после инструкции try:

```
void g()
{
  try {
    while (true)
      new int[10000000];
  } // Пример правильного порядка !!!
  catch (const std::bad_alloc & e)
  { cout << "Мало памяти: " << e.what() << '\n'; }
  catch (std::exception & e)
  { /*обработка любого исключения стандартной библиотеки*/ }
  catch (...) { /* обработка любого исключения */ }
}
```

Обработчики проверяются по порядку. Поэтому **если прописать их в обратном порядке**, то вариант с **exception** никогда не будет рассматриваться. Даже если убрать обработчик, перехватывающий все исключения, то **bad_alloc** по прежнему никогда не рассматривается, так как он является производным от exception.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перехват всех исключений

Многоточие «…» означает «любой аргумент». Поэтому catch(...) означает «перехват всех исключений»:

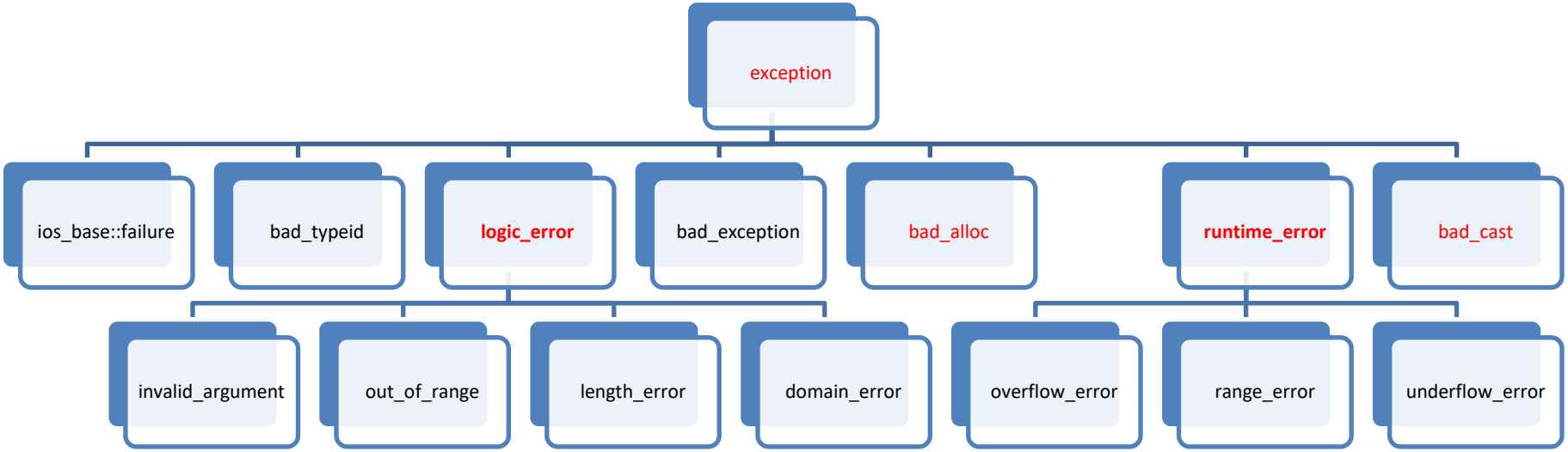
```
void m() {
  try {
    /* что-нибудь*/
  }
  catch(...) {
    // очистка
    throw;
  }
}
```

То есть, если в результате выполнения m() возникает исключение, обработчик осуществляет очистку. После того, как локальная очистка завершена, приведшее к ней исключение повторно генерируется для дальнейшей обработки ошибки.

Такой обработчик во многих важных случаях является не самым лучшим решением.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Стандартные классы исключений



© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Основные классы исключений

- **exception** – базовый класс для всех исключений, выбрасываемых кодом библиотеки;
- **logic_error** – базовый класс для ошибок, которые можно было бы выявить до выполнения программы;
- **runtime_error** – базовый класс ошибок, которые, как правило, можно выявить только во время работы программы;
- **bad_alloc** – ошибка выделения памяти;
- **bad_cast** – ошибка приведения типа;
- **invalid_argument** – ошибка в значении аргумента функции (например, при вводе).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Механизм RTTI (Run-Time Type Identification)

Динамическая идентификация типа данных (RTTI) – механизм, который позволяет определить тип данных переменной или объекта во время выполнения программы.

Механизм RTTI состоит из трех частей:

1. Операция dynamic_cast. В основном предназначена для получения указателя на объект производного класса при наличии указателя на объект полиморфного базового класса;

2. Операция typeid. Служит для идентификации точного типа объекта при наличии указателя на полиморфный базовый класс;

3. Структура type_info. Позволяет получить дополнительную информацию, ассоциированную с типом.

Для использования RTTI в программу следует включить заголовок **<typeinfo>**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Приведение типов (dynamic_cast)

Операция dynamic_cast реализует приведение типов (указателей или ссылок) полиморфных классов в динамическом режиме.

Синтаксис использования операции dynamic_cast :

```
dynamic_cast <целевой_тип> (выражение)
```

Если даны **два полиморфных класса B и D** (причем D – производный от B), то dynamic_cast всегда может привести D* к B*.

Также dynamic_cast может привести B* к D*, но только в том случае, если объект, на который указывает указатель, действительно является объектом типа D (либо производным от него)!

При неудачной попытке приведения типов результатом выполнения dynamic_cast является 0, если в операции использовались указатели.

Если же в операции использовались ссылки, генерируется исключение типа **bad_cast**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример приведения типов (dynamic_cast)

```
// Base – полиморфный класс (есть virtual)
class Base {virtual void f() {} ... };
class Derived : public Base { ... };

int main()
{
    system("chcp 1251");
    // Указатель и объект базового класса
    Base *bp,  b_ob;
    // Указатель и объект производного класса
    Derived *dp, d_ob;

    bp = &d_ob; // Фактически хранит производный
    dp = dynamic_cast<Derived *>(bp);
    if(dp)
        cout << "Приведение типов прошло успешно\n";

    bp = &b_ob; // Фактически хранит базовый
    dp = dynamic_cast<Derived *>(bp);
    if(!dp)
        cout << "Приведения типов не произошло\n";
    return 0;
}
```

```
Приведение типов прошло успешно
Приведения типов не произошло
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Динамическая идентификация типов (RTTI)

Динамическая идентификация типов позволяет установить фактический производный тип объекта, адресуемого по указателю на базовый полиморфный класс. Для выяснения фактического производного типа служит унарная операция typeid, операнд которой должен иметь тип класса хотя бы с одним виртуальным методом. Синтаксис использования операции typeid:

typeid (объект)

Операция typeid возвращает ссылку на объект класса type_info, в котором определены следующие открытые члены:

```
bool operator == (const type_info & объект);
bool operator != (const type_info & объект);
bool before(const typeinfo &ob) const; //выше в иерар?
const char* name(); //возвращает указатель на имя типа
```

typeid часто применяется к разыменованным указателям (typeid (*p)). Если указатель на полиморфный класс p == NULL, то будет сгенерировано исключение типа **bad_typeid**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример

```
class Base { virtual void f() {} ... };
class Derived1: public Base { ... };
class Derived2: public Base { ... };

int main()
{
    int i;
    Base *p, b_ob;
    Derived1 ob1;
    Derived2 ob2;

    cout << "Тип i - " << typeid(i).name();
    p = &b_ob;
    cout << "\np указывает на объект типа " << typeid(*p).name();
    p = &ob1;
    cout << "\np указывает на объект типа " << typeid(*p).name();
    p = &ob2;
    cout << "\np указывает на объект типа " << typeid(*p).name();

    if( typeid(ob1) == typeid(ob2) )
        cout << "\nТип объектов ob1 и ob2 одинаков\n";
    else
        cout << "\nТип объектов ob1 и ob2 не одинаков\n";
    return 0;
}
```

```
Тип i - int
p указывает на объект типа class Base
p указывает на объект типа class Derived1
p указывает на объект типа class Derived2
Тип объектов ob1 и ob2 не одинаков
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева